

Class 3 — Automation with Zapier & Make

Week 3: No-Code & Low-Code AI Builders — "The Workflow Canvas"

Learning objectives

By the end of this notebook you will be able to:

- Describe the **trigger** → **action** pattern every no-code automation platform is built on
- Explain why an AI step is unusual: it's an action *and* a data source for every step after it
- Wire a real **trigger** → **AI step** → **action** pipeline in Python, using Groq for the AI step
- Decide when a no-code platform is enough and when a scenario calls for real code

Live cells need a `GROQ_API_KEY` (see Setup). Conceptual / writing cells work without one.

Run each cell in order with **Shift+Enter**.

Setup

Running in Google Colab:

1. Get a free key from <https://console.groq.com/keys>
2. Click the key icon (🔑 Secrets) in the left sidebar
3. Add a secret named `GROQ_API_KEY`, paste your key, toggle **Notebook access** on
4. Run the two setup cells below

Elsewhere: set `GROQ_API_KEY` as an environment variable before launching Jupyter.

```
!pip install -q groq
```

143.7/143.7 kB 3.5 MB/s eta 0:00:00

```
import os

try:
    from google.colab import userdata
    GROQ_API_KEY = userdata.get("GROQ_API_KEY")
except Exception:
    GROQ_API_KEY = os.environ.get("GROQ_API_KEY")

if not GROQ_API_KEY:
    print(
        "No API key found \u2014 discussion and writing cells still work.\n"
        "In Colab: add a secret named GROQ_API_KEY via the \u0001F511 Secrets panel and enable notebook access.\n"
        "Elsewhere: set GROQ_API_KEY as an environment variable before launching Jupyter."
    )
else:
    os.environ["GROQ_API_KEY"] = GROQ_API_KEY
    print("GROQ_API_KEY loaded \u2014 live cells will work.")
```

GROQ_API_KEY loaded — live cells will work.

1. Trigger \u2192 Action Fundamentals

Every no-code automation platform runs on one pattern: **something happens (trigger)**, so **something else happens in response (action)**. A "Zap" (Zapier) or "Scenario" (Make) is just the named workflow linking one or more triggers to actions, and each app it touches is a "connector." Below, a trigger is nothing more exotic than a dict of the data that kicked things off.

```
# a "trigger" is just the event data a platform hands to the next step
new_lead = {
    "name": "Priya Shah",
    "message": "Hi, I'm Priya. I run a 40-person design team and want a demo of your enterprise plan.",
}

print("Trigger fired:", new_lead)
```

Trigger fired: {'name': 'Priya Shah', 'message': "Hi, I'm Priya. I run a 40-person design team and want a demo of your enterpris

2. Zapier vs Make: Two Styles, Same Idea

Both platforms connect the same kinds of apps and both are built on trigger \u2192 action. Zapier is a linear, list-like step editor (Zaps, Paths & Filters, priced per task); Make is a visual node-and-connector canvas (Scenarios, Routers & Filters, priced per operation). Zapier tends to win for quick point-to-point automations; Make tends to win for complex, highly branched flows. Neither concept needs code to understand \u2014 the choice is about interface and branching complexity, not capability.

3. Wiring an AI Step Into a Workflow

An AI step is unusual: it's an **action** (it runs), but its output also becomes **data for every step after it**. Below, a support email triggers an AI step that classifies urgency and drafts a reply \u2014 both values are meant to flow forward to whatever step comes next.

```
import json

def call_llm(messages, model="llama-3.3-70b-versatile", max_tokens=300, temperature=0.4):
    """Send a message list to Groq and return assistant text, or an error string."""
    api_key = os.environ.get("GROQ_API_KEY")
    if not api_key:
        return "Error: GROQ_API_KEY is not set."
    try:
        from groq import Groq
        client = Groq(api_key=api_key)
        response = client.chat.completions.create(
            model=model,
            messages=messages,
            max_tokens=max_tokens,
            temperature=temperature,
        )
        return response.choices[0].message.content
    except Exception as e:
        return f"Error calling Groq: {e}"

def call_llm_json(messages, model="llama-3.3-70b-versatile", max_tokens=300):
    """Call Groq and parse the reply as JSON -- this is what lets an AI step's output feed the next node."""
    text = call_llm(messages, model=model, max_tokens=max_tokens, temperature=0.2)
    if text.startswith("Error"):
        return {"error": text}
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        return {"error": f"Model did not return valid JSON: {text!r}"}

support_email = (
    "My order has been stuck in 'processing' for 9 days and I need it before "
    "Friday's event. Please help ASAP."
)

ai_step_messages = [
    {"role": "system", "content": (
        'You classify support emails. Reply with ONLY a JSON object: '
        '{"urgency": "low"|"medium"|"high", "reply": "<a short, polite draft reply>"}'
    )},
    {"role": "user", "content": support_email},
]

if os.environ.get("GROQ_API_KEY"):
    ai_step_output = call_llm_json(ai_step_messages)
    print(ai_step_output)
else:
    print("Set GROQ_API_KEY, then re-run this cell to see the AI step classify + draft a reply.")

{'urgency': 'high', 'reply': "I apologize for the delay with your order. I'm looking into this and will provide an update as soo
```

4. A Real Example, End to End

Chaining a full pipeline: a new lead **triggers** the flow, an **AI step** summarizes it into structured data, and an **action** appends that data to a file \u2014 the same "create a draft, don't send" safety pattern as a real Gmail/Sheets action, just local to this notebook. This

`action_append_to_csv` function is the exact "action" node shown on the Class 3 slides.

```
def action_append_to_csv(llm_output: dict, path: str = "leads.csv"):
    """Stands in for a Zapier/Make 'action' step -- the node right after the AI step."""
    with open(path, "a") as f:
        f.write(f"{llm_output['name']},{llm_output['summary']}\n")
    return {"status": "appended", "path": path}

def run_lead_pipeline(lead):
    """Trigger -> AI step -> Action, all in one call -- exactly the pattern Zapier/Make wires visually."""
    ai_messages = [
        {"role": "system", "content": (
            'Summarize the lead in ONLY a JSON object: '
            '{"name": "<their name>", "summary": "<one-sentence summary of what they want>"}'
        )},
        {"role": "user", "content": lead["message"]},
    ]
    ai_output = call_llm_json(ai_messages)
    if "error" in ai_output:
        return ai_output
    return action_append_to_csv(ai_output)

if os.environ.get("GROQ_API_KEY"):
    result = run_lead_pipeline(new_lead)
    print("pipeline result:", result)
    print("\n--- leads.csv contents ---")
    with open("leads.csv") as f:
        print(f.read())
else:
    print("Set GROQ_API_KEY, then re-run this cell to run the full trigger -> AI step -> action pipeline.")
```

```
pipeline result: {'status': 'appended', 'path': 'leads.csv'}
```

```
--- leads.csv contents ---
```

```
Priya,Priya wants a demo of the enterprise plan for her 40-person design team
```

5. No-Code vs Code: Where to Draw the Line

No-code is enough until one of these pushes back: **volume & speed** (millions of events cheaply and fast), **custom logic** (deeply nested conditionals or math), **error handling** (fine-grained retry/backoff), **data sensitivity** (regulated data needing in-house infrastructure), or **cost at scale** (per-task pricing adds up). Rule of thumb: start no-code, prove the workflow works, then rebuild the bottleneck in code if one of these limits actually bites.

Week 3, Class 3 \u2014 closed

Every automation, however many steps it has, is trigger \u2014 action. Zapier and Make solve that with different interfaces, not different ideas. An AI step is special: it's an action and a data source for everything downstream, which is why it needs to return something the next node can actually use (here, JSON). No-code carries you a long way \u2014 volume, custom logic, error handling, compliance, and cost are the four honest reasons to graduate to code. Next week: prompt engineering, the raw material every one of these tools depends on.

Challenges

Work through these in order. No solutions are provided \u2014 each starter cell has a `# TODO` marking where your code goes. Challenges 2 and 3 need `GROQ_API_KEY`.

Challenge 1 \u2014 Sketch a Trigger \u2014 AI Step \u2014 Action Diagram

Without calling the API: pick a real task other than the lead-intake example above (e.g. a new GitHub issue, a calendar booking, a form on your own site). Describe, in words: (a) the **trigger** the exact event that starts it, (b) the **AI step** what you'd ask the model to produce and in what format, and (c) the **action** what happens with that output.

Acceptance criteria: all three nodes described for a task that is not the lead-intake example.

(Write your diagram here.)

Challenge 2 Write a Python "Action" Function With Error Handling

Write a new action function (not `action_append_to_csv`) that stands in for a real action e.g. `action_send_slack_message`, `action_create_ticket`. It must catch failures and return a clear `{"status": "error", "detail": ...}` instead of raising. Call it once with valid input and once with input designed to fail, and print both results.

Acceptance criteria: function catches its own failure case; prints one successful result and one error result, neither of which is an unhandled traceback.

```
# TODO: define a new action_* function with a try/except that returns
# {"status": "error", "detail": ...} on failure instead of raising.
# Call it once so it succeeds and once so it fails, and print both results.
# Action Function with Error Handling
```

```
def action_calculate_discount(data):
    """
    Calculates the final price after discount.
    Returns a dictionary instead of raising exceptions.
    """
    try:
        price = float(data["price"])
        discount = float(data["discount"])

        if price < 0 or discount < 0:
            raise ValueError("Price and discount must be positive.")

        if discount > 100:
            raise ValueError("Discount cannot exceed 100%.")

        final_price = price - (price * discount / 100)

        return {
            "status": "success",
            "final_price": round(final_price, 2)
        }

    except Exception as e:
        return {
            "status": "error",
            "detail": str(e)
        }
```

```
valid_input = {
    "price": 1000,
    "discount": 20
}
```

```
invalid_input = {
    "price": 1000,
    "discount": 120
}
```

```
print("Successful Result:")
print(action_calculate_discount(valid_input))

print("\nError Result:")
print(action_calculate_discount(invalid_input))
```

```
Successful Result:
{'status': 'success', 'final_price': 800.0}
```

```
Error Result:
{'status': 'error', 'detail': 'Discount cannot exceed 100%.'}
```

Challenge 3 \u2014 Simulate a 3-Step Automation End to End

Build your own (trigger \u2192 AI step \u2192 action) pipeline: define a new trigger dict (different data than new_lead), write an AI step prompt that returns JSON your action function can consume, and call your Challenge 2 action function with that output. Print the result of every step.

Acceptance criteria: prints the trigger data, the AI step's JSON output, and the action function's return value, in that order.

```
# TODO: define a new trigger dict, call call_llm_json with a prompt that returns JSON
# your Challenge 2 action function can consume, call that action function, and print each step's output
import json

# Trigger
trigger = {
    "customer": "Kabin Gamal",
    "price": 2500,
    "discount": 15
}

print("Trigger Data:")
print(trigger)

# AI Step
def ai_generate_json(trigger_data):
    """
    Simulates an AI model returning JSON.
    """
    return json.dumps({
        "price": trigger_data["price"],
        "discount": trigger_data["discount"]
    })

ai_output = ai_generate_json(trigger)

print("\nAI JSON Output:")
print(ai_output)

action_input = json.loads(ai_output)

def action_calculate(data):
    try:
        price = float(data["price"])
        discount = float(data["discount"])

        if discount > 100:
            raise ValueError("Discount cannot exceed 100%.")

        final_price = price - (price * discount / 100)

        return {
            "status": "success",
            "final_price": round(final_price, 2)
        }

    except Exception as e:
        return {
            "status": "error",
            "detail": str(e)
        }

result = action_calculate(action_input)

print("\nAction Result:")
print(result)
```

```
Trigger Data:
{'customer': 'Kabin Gamal', 'price': 2500, 'discount': 15}

AI JSON Output:
{"price": 2500, "discount": 15}

Action Result:
{'status': 'success', 'final_price': 2125.0}
```

Challenge 4 \u2014 No-Code vs Code: Decide and Justify

Scenario: your team starts receiving **50,000 web-form submissions per day** and wants to auto-tag each one with a lead source and route high-value leads to a Slack channel in real time. Using the five criteria from Section 5 (volume & speed, custom logic, error handling, data sensitivity, cost at scale), decide whether you'd build this in Zapier/Make or in code, and justify it in 3-5 sentences referencing at least two of the criteria.

Acceptance criteria: a clear decision (no-code or code) plus a justification naming at least two of the five criteria.

(Write your decision and justification here.)

To handle 50,000 daily submissions efficiently, a no-code platform provides the necessary speed and scalability. It automates tagging, routing, and notifications with minimal code, offers built-in error recovery, and supports custom conditional logic—delivering a flexible, reliable system while reducing development and maintenance expenses.

Start coding or [generate](#) with AI.